

서버 배포 완전 가이드

Git · Docker · CI/CD · Nginx 를 활용한
프로덕션 배포 플로우 완전 정복

작성일 2025년

대상 백엔드 / 풀스택 개발자

버전 v1.0

목차

01 배포 플로우 개요

02 로컬 개발 환경 세팅

- Docker Desktop 설치 및 구성
- Dockerfile 작성법
- docker-compose.yml (로컬 개발용)
- .env 파일 관리

03 Git Push 및 버전 관리

- 브랜치 전략 (Git Flow)
- .gitignore 설정

04 CI/CD 파이프라인

- GitHub Actions 워크플로우
- 자동 테스트 설정

05 Docker 이미지 빌드 & 테스트

- 멀티스테이지 빌드
- 이미지 최적화 전략

06 Container Registry

07 서버 배포

- Docker Compose (소규모)
- Kubernetes (대규모)

08 Nginx 리버스 프록시

- 설정 파일 작성
- SSL/HTTPS 적용 (Let's Encrypt)

09 전체 흐름 요약

배포 플로우 개요

현대적인 웹 서비스 배포는 단순히 서버에 파일을 올리는 것을 넘어, **코드 작성부터 사용자 응답까지** 일련의 자동화된 단계로 이루어집니다. 이 가이드에서는 실무에서 가장 널리 쓰이는 Git → CI/CD → Docker → Nginx 기반의 배포 파이프라인 전체를 단계별로 설명합니다.

전체 배포 파이프라인

- 1 로컬 개발 → 코드 작성, Dockerfile 준비
- 2 Git Push → GitHub / GitLab 원격 저장소
- 3 CI/CD 트리거 → GitHub Actions / Jenkins 자동 실행
- 4 Docker 빌드 & 테스트 → 이미지 생성 + 자동 테스트
- 5 Container Registry Push → Docker Hub / AWS ECR
- 6 서버 배포 → Docker Compose / Kubernetes
- 7 Nginx 리버스 프록시 → SSL, 도메인 라우팅
- 8 사용자 응답 → HTTPS로 최종 서비스 제공

핵심 원칙

이 파이프라인의 가장 중요한 특징은 **자동화**입니다. 개발자가 코드를 push하면, 이후 단계는 모두 자동으로 실행되어 사람의 개입 없이 프로덕션까지 배포됩니다. 이를 **CD (Continuous Deployment)**라고 합니다.

SECTION 02

로컬 개발 환경 세팅

프로덕션 배포의 핵심 철학은 "내 컴퓨터에서는 되는데"를 없애는 것입니다. Docker를 사용하면 로컬 환경과 서버 환경을 동일하게 만들 수 있습니다.

2-1. Docker Desktop 설치

Docker Desktop은 로컬에서 컨테이너를 실행하는 핵심 도구입니다. **macOS / Windows**에서는 Docker Desktop을, **Linux**에서는 Docker Engine을 설치합니다.

```
# macOS (Homebrew)
brew install --cask docker

# Ubuntu / Debian
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
newgrp docker

# 설치 확인
docker --version          # Docker version 26.x.x
docker compose version    # Docker Compose version v2.x.x
```

2-2. 프로젝트 디렉토리 구조

배포를 염두에 둔 프로젝트 기본 구조입니다.

```
my-app/
├─ src/                # 애플리케이션 소스코드
├─ tests/              # 테스트 코드
├─ Dockerfile          # 프로덕션 이미지 빌드 설명서
├─ docker-compose.yml  # 로컬 개발 환경 구성
├─ docker-compose.prod.yml # 프로덕션 환경 구성
├─ .env.example        # 환경변수 예시 (Git에 커밋)
├─ .env                # 실제 환경변수 (Git에 절대 커밋 X)
├─ .gitignore
└─ nginx/
    └─ default.conf    # Nginx 설정 파일
```

2-3. Dockerfile 작성

Dockerfile은 "이 앱을 어떤 환경에서 어떻게 실행할지" 적어 두는 설계도입니다. 멀티스테이지 빌드를 활용하면 최종 이미지 크기를 크게 줄일 수 있습니다.

Node.js 예시 (멀티스테이지 빌드)

```
# — 1단계: 빌드 환경 —————
FROM node:20-alpine AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci                # npm install 보다 엄격/빠름
COPY . .
RUN npm run build

# — 2단계: 실행 환경 (훨씬 가벼움) —————
FROM node:20-alpine AS runner

WORKDIR /app
ENV NODE_ENV=production

# 빌드 결과물만 복사 (소스코드, node_modules 제외)
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/package*.json ./
RUN npm ci --only=production

# 보안: root가 아닌 일반 유저로 실행
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

EXPOSE 3000
CMD ["node", "dist/index.js"]
```

Python (FastAPI) 예시

```
FROM python:3.12-slim

WORKDIR /app

# 의존성 먼저 복사 (캐시 레이어 활용)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

TIP — 레이어 캐시 활용

`COPY package.json .` 후 `RUN npm install` 을 먼저 실행하면, 소스코드가 변경되어도 의존성 설치 레이어가 캐시되어 빌드 속도가 크게 빨라집니다.

니다. 자주 변하는 파일은 Dockerfile 하단에 배치하세요.

2-4. docker-compose.yml (로컬 개발용)

docker-compose는 여러 컨테이너(앱 서버, DB, Redis 등)를 한 번에 실행하는 도구입니다. 로컬에서 프로덕션과 유사한 환경을 구성할 수 있습니다.

```
version: '3.9'

services:
  # 애플리케이션 서버
  app:
    build:
      context: .
      target: builder      # 개발용: 빌드 스테이지 사용
    ports:
      - "3000:3000"
    volumes:
      - ../app              # 소스코드 실시간 반영 (핫리로드)
      - /app/node_modules  # node_modules는 컨테이너 것 사용
    environment:
      - NODE_ENV=development
    env_file:
      - .env                # 환경변수 파일 로드
    depends_on:
      db:
        condition: service_healthy # DB 준비 완료 후 시작

# 데이터베이스
db:
  image: postgres:16-alpine
  environment:
    POSTGRES_DB: myapp
    POSTGRES_USER: ${DB_USER}
    POSTGRES_PASSWORD: ${DB_PASSWORD}
  volumes:
    - postgres_data:/var/lib/postgresql/data # 데이터 영속화
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${DB_USER}"]
    interval: 5s
    timeout: 5s
    retries: 5

# 캐시 서버
redis:
  image: redis:7-alpine
  ports:
    - "6379:6379"

volumes:
  postgres_data:
```

```
# 로컬 개발 실행 명령어
docker compose up -d          # 백그라운드 실행
docker compose logs -f app    # 로그 확인
docker compose down           # 종료
docker compose down -v        # 볼륨까지 삭제 (DB 초기화)
```

2-5. 환경변수 (.env) 관리

주의

`.env` 파일에는 DB 비밀번호, API 키 등 민감한 정보가 담깁니다. 반드시 `.gitignore`에 추가하여 Git에 올라가지 않도록 해야 합니다.

```
# .env.example (Git에 커밋 — 팀원들이 참고)
DATABASE_URL=postgresql://user:password@db:5432/myapp
REDIS_URL=redis://redis:6379
JWT_SECRET=your-secret-key-here
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=

# .gitignore
.env
.env.local
.env.production
```


Git Push 및 버전 관리

코드를 원격 저장소에 push하는 순간, CI/CD 파이프라인이 자동으로 시작됩니다. 따라서 어떤 브랜치에 push하느냐가 배포 흐름을 결정합니다.

3-1. 브랜치 전략 (Git Flow)

브랜치	용도	배포 대상
main	프로덕션 배포 기준	프로덕션 서버
develop	개발 통합 브랜치	스테이징 서버
feature/*	기능 개발	CI 검사만 수행
hotfix/*	긴급 버그 수정	즉시 프로덕션

```
# 일반적인 개발 흐름
git checkout -b feature/user-auth    # 기능 브랜치 생성
git add .
git commit -m "feat: 사용자 인증 기능 추가"
git push origin feature/user-auth    # CI 테스트 실행됨

# PR 승인 후 develop에 머지
git checkout develop
git merge feature/user-auth          # 스테이징 배포됨

# QA 통과 후 main에 머지 → 프로덕션 자동 배포
git checkout main
git merge develop
```

CI/CD 파이프라인

CI (Continuous Integration)는 코드가 push될 때마다 자동으로 테스트와 빌드를 실행하는 것이고, CD (Continuous Delivery/Deployment)는 테스트가 통과하면 자동으로 서버에 배포하는 것입니다.

4-1. GitHub Actions 워크플로우 예시

프로젝트 루트의 `.github/workflows/deploy.yml` 파일에 작성합니다.

```
name: Build and Deploy

# main 브랜치에 push될 때만 실행
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${github.repository}

jobs:
  # — 1단계: 테스트 —————
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Run linter
        run: npm run lint

      - name: Run tests
        run: npm test -- --coverage

  # — 2단계: Docker 빌드 & Push —————
  build-and-push:
    needs: test # 테스트 통과 후에만 실행
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'

    steps:
      - uses: actions/checkout@v4
```

```
- name: Log in to Container Registry
  uses: docker/login-action@v3
  with:
    registry: ${ env.REGISTRY }
    username: ${ github.actor }
    password: ${ secrets.GITHUB_TOKEN }

- name: Build and push Docker image
  uses: docker/build-push-action@v5
  with:
    context: .
    push: true
    tags: |
      ${ env.REGISTRY }/${ env.IMAGE_NAME }:latest
      ${ env.REGISTRY }/${ env.IMAGE_NAME }:${ github.sha }
```

— 3단계: 서버 배포 —————

```
deploy:
  needs: build-and-push
  runs-on: ubuntu-latest

  steps:
    - name: Deploy to server via SSH
      uses: appleboy/ssh-action@v1
      with:
        host: ${ secrets.SERVER_HOST }
        username: ${ secrets.SERVER_USER }
        key: ${ secrets.SSH_PRIVATE_KEY }
        script: |
          cd /opt/myapp
          docker compose pull
          docker compose up -d --remove-orphans
          docker image prune -f
```

Docker 이미지 빌드 & 테스트

CI/CD 파이프라인 안에서 Docker 이미지가 빌드됩니다. 이 단계에서 **테스트 실패** → **빌드 중단**이 이루어져, 버그가 있는 코드는 절대 프로덕션에 올라가지 않습니다.

5-1. 이미지 최적화 체크리스트

- **Alpine 베이스 이미지 사용** — `node:20` (1.1GB) 대신 `node:20-alpine` (약 180MB)
- **멀티스테이지 빌드** — 빌드 도구를 최종 이미지에서 제외
- **.dockerignore 설정** — 불필요한 파일 제외
- **레이어 수 최소화** — 여러 RUN 명령을 &&로 연결
- **non-root 유저 실행** — 보안 강화

```
# .dockerignore
node_modules
.git
.gitignore
*.md
.env*
coverage
.nyc_output
dist
.DS_Store
Dockerfile*
docker-compose*
.github
```

5-2. 로컬에서 이미지 테스트

```
# 이미지 빌드
docker build -t myapp:local .

# 컨테이너 실행 및 테스트
docker run --rm -p 3000:3000 --env-file .env myapp:local

# 이미지 크기 확인
docker images myapp

# 컨테이너 내부 확인
docker run --rm -it myapp:local sh
```

Container Registry

빌드된 Docker 이미지를 저장하는 원격 저장소입니다. 마치 Git이 코드를 저장하듯, Registry는 이미지를 버전별로 저장합니다.

Registry	특징	적합한 경우
Docker Hub	가장 유명, 무료 플랜 있음	오픈소스, 개인 프로젝트
AWS ECR	AWS 서비스와 통합 용이	AWS 인프라 사용 시
GCP GCR	GCP와 통합	GCP 인프라 사용 시
GitHub GHCR	GitHub Actions와 무료 통합	GitHub 기반 프로젝트

```
# Docker Hub에 Push
docker login
docker tag myapp:local username/myapp:v1.0.0
docker push username/myapp:v1.0.0
docker push username/myapp:latest

# GitHub Container Registry에 Push
echo $GITHUB_TOKEN | docker login ghcr.io -u USERNAME --password-stdin
docker tag myapp:local ghcr.io/username/myapp:latest
docker push ghcr.io/username/myapp:latest
```

TIP — 이미지 태그 전략

latest 태그 외에 Git 커밋 해시(:abc1234)나 버전(:v1.2.3)을 함께 태그하면, 문제 발생 시 이전 버전으로 즉시 롤백할 수 있습니다.

서버 배포

7-1. Docker Compose (소규모 / 단일 서버)

개인 프로젝트나 소규모 서비스에 적합합니다. 서버에 SSH로 접속해서 실행합니다.

```
# docker-compose.prod.yml (서버용)
version: '3.9'

services:
  app:
    image: ghcr.io/username/myapp:latest # 로컬 빌드 X, Registry에서 pull
    restart: always # 서버 재시작 시 자동 실행
    expose:
      - "3000" # 외부 노출 X, Nginx만 접근
    env_file:
      - .env.production
    networks:
      - app-network

  db:
    image: postgres:16-alpine
    restart: always
    volumes:
      - postgres_data:/var/lib/postgresql/data
    env_file:
      - .env.production
    networks:
      - app-network

  nginx:
    image: nginx:alpine
    restart: always
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
      - /etc/letsencrypt:/etc/letsencrypt:ro # SSL 인증서
    depends_on:
      - app
    networks:
      - app-network

networks:
  app-network:
    driver: bridge

volumes:
  postgres_data:
```

```
# 서버에서 실행
docker compose -f docker-compose.prod.yml pull    # 새 이미지 다운로드
docker compose -f docker-compose.prod.yml up -d    # 무중단 재시작
docker compose -f docker-compose.prod.yml ps       # 상태 확인
```

7-2. Kubernetes (대규모 / 멀티 서버)

트래픽이 많거나 고가용성이 필요한 경우 사용합니다.

```
# deployment.yaml - 앱 배포 정의
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 3                                # 3개 인스턴스 운영
  strategy:
    type: RollingUpdate                      # 무중단 배포
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  selector:
    matchLabels:
      app: myapp
  template:
    spec:
      containers:
        - name: myapp
          image: ghcr.io/username/myapp:latest
          ports:
            - containerPort: 3000
      resources:
        requests:
          memory: "128Mi"
          cpu: "100m"
        limits:
          memory: "512Mi"
          cpu: "500m"
```

Nginx 리버스 프록시

Nginx는 배포 아키텍처에서 **문지기(게이트웨이)** 역할을 합니다. 외부에서 들어오는 80(HTTP), 443(HTTPS) 포트를 받아 내부 앱 컨테이너로 전달합니다.

Nginx가 처리하는 것들

- **SSL/TLS 종단** — HTTPS 복호화를 Nginx에서 처리, 앱은 HTTP만 다룸
- **도메인 라우팅** — `api.myapp.com` → 백엔드, `myapp.com` → 프론트엔드
- **로드밸런싱** — 여러 앱 인스턴스에 트래픽 분산
- **정적 파일 서빙** — 이미지, CSS, JS 파일 직접 제공
- **Rate Limiting** — 과도한 요청 차단

8-1. Nginx 설정 파일


```

# /nginx/default.conf

# HTTP → HTTPS 리다이렉트
server {
    listen 80;
    server_name myapp.com www.myapp.com;
    return 301 https://$host$request_uri;
}

# HTTPS 메인 서버
server {
    listen 443 ssl http2;
    server_name myapp.com www.myapp.com;

    # SSL 인증서 경로 (Let's Encrypt)
    ssl_certificate      /etc/letsencrypt/live/myapp.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/myapp.com/privkey.pem;

    # 보안 헤더
    add_header X-Frame-Options "SAMEORIGIN";
    add_header X-Content-Type-Options "nosniff";
    add_header Strict-Transport-Security "max-age=31536000" always;

    # 요청 크기 제한
    client_max_body_size 10M;

    # 리버스 프록시 설정
    location / {
        proxy_pass          http://app:3000;    # Docker 내부 네트워크
        proxy_http_version  1.1;
        proxy_set_header    Upgrade $http_upgrade;
        proxy_set_header    Connection 'upgrade';
        proxy_set_header    Host $host;
        proxy_set_header    X-Real-IP $remote_addr;
        proxy_set_header    X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header    X-Forwarded-Proto $scheme;
        proxy_cache_bypass  $http_upgrade;
    }

    # 정적 파일 직접 서빙 (성능 향상)
    location /static/ {
        alias /app/static/;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }
}

```

8-2. SSL 인증서 발급 (Let's Encrypt — 무료)

```
# Certbot 설치 및 인증서 발급
sudo apt install certbot python3-certbot-nginx

sudo certbot --nginx -d myapp.com -d www.myapp.com

# 자동 갱신 (90일마다 갱신 필요 → cron 등록)
sudo certbot renew --dry-run    # 갱신 테스트
# crontab -e 에 추가:
# 0 0 * * * certbot renew --quiet
```

전체 흐름 요약

1 로컬 개발

개발자 PC

Dockerfile 작성, docker-compose로 로컬 실행, `.env` 로 환경변수 관리

2 Git Push

GitHub / GitLab

코드를 원격 저장소에 push → 브랜치 전략에 따라 CI/CD 트리거 결정

3 CI/CD 파이프라인

GitHub Actions

자동으로 린트, 테스트 실행 → 실패 시 파이프라인 중단, 슬랙 알림

4 Docker 이미지 빌드

CI 서버

Dockerfile로 이미지 빌드, 멀티스테이지로 크기 최소화

5 Registry Push

Docker Hub / ECR

빌드된 이미지를 원격 레지스트리에 push, 버전 태그 부착

6 서버 배포

운영 서버

SSH로 서버 접속, `docker compose pull && up -d` 로 무중단 배포

7 Nginx 리버스 프록시

80 / 443 포트

SSL 처리, 도메인 라우팅, 앱 컨테이너 내부 포트로 트래픽 전달

핵심 정리

위 7단계가 완전히 자동화되면, 개발자는 코드를 **push**하는 것만으로 프로덕션 배포가 완료됩니다.

- Docker: 환경 차이 문제 제거 ("내 컴퓨터에서는 됩니다" 해결)
- CI/CD: 버그 있는 코드의 프로덕션 진입 차단
- Nginx: HTTPS 보안, 유연한 트래픽 관리
- Container Registry: 이미지 버전 관리 및 롤백 지원